DEMYSTIFYING EMBEDDING SPACES USING LARGE LANGUAGE MODELS

Guy Tennenholtz*, Yinlam Chow, Chih-Wei Hsu, Jihwan Jeong, Lior Shani, Azamat Tulepbergenov, Deepak Ramachandran, Martin Mladenov, Craig Boutilier

Google Research

arXiv:2310.04475v2 [cs.CL] 13 Mar 2024

ABSTRACT

Embeddings have become a pivotal means to represent complex, multi-faceted information about entities, concepts, and relationships in a condensed and useful format. Nevertheless, they often preclude direct interpretation. While downstream tasks make use of these compressed representations, meaningful interpretation usually requires visualization using dimensionality reduction or specialized machine learning interpretability methods. This paper addresses the challenge of making such embeddings more interpretable and broadly useful, by employing large language models (LLMs) to directly interact with embeddings – transforming abstract vectors into understandable narratives. By injecting embeddings into LLMs, we enable querying and exploration of complex embedding data. We demonstrate our approach on a variety of diverse tasks, including: enhancing concept activation vectors (CAVs), communicating novel embedded entities, and decoding user preferences in recommender systems. Our work couples the immense information potential of embeddings with the interpretative power of LLMs.

1 INTRODUCTION

The success of deep learning has brought forth a paradigm shift in knowledge representation through the concept of *embeddings*—dense vector representations that capture high-dimensional information about entities, concepts, or relationships in a compact and useful format. Embeddings are ubiquitous, finding application in natural language processing (Mikolov et al., 2013; Devlin et al., 2018), recommender systems (Rendle, 2010), protein sequence modeling (Rives et al., 2021), and more. These embeddings are invaluable, capturing nuanced relationships and semantic structure in data which traditional machine learning (ML) approaches often miss. Nevertheless, understanding these abstract representations remains challenging.

By design, the structure and underlying information carried by an embedding is heavily adapted to the idiosyncrasies of the downstream task, posing a substantial challenge to its interpretation and manipulation. Previous work on ML interpretability offers various task-independent means to interpret embeddings, including dimensionality reduction techniques (e.g., *t-SNE* (Van der Maaten & Hinton, 2008), *UMAP* (McInnes et al., 2018)) or *concept activation vectors (CAVs)* (Kim et al., 2018). While very useful, these techniques are fairly narrow in scope.

As an alternative to such interpretability methods, suppose one could engage with embeddings using natural language to query information not directly expressed by the name or description of the underlying entity/concept. Perhaps one could even extract information from the embedding representations of *non-existent or hypothetical* entities. Consider, for instance, an embedding space representing items from an online commerce site, trained using a large corpus of user ratings or purchases, reviews, and other multi-faceted data sources. The embedding representation of an item may *implicitly* embody intricate details about its quality, usability, design, customer satisfaction, etc. Moreover, suppose one wanted to understand the properties of a *hypothetical* item at a specific point in the embedding space, say, if we predicted a potential market for such an item. Since no such item (or description) exists, asking a conventional language model to describe this embedding point would

*Correspondence to: guytenn@gmail.com

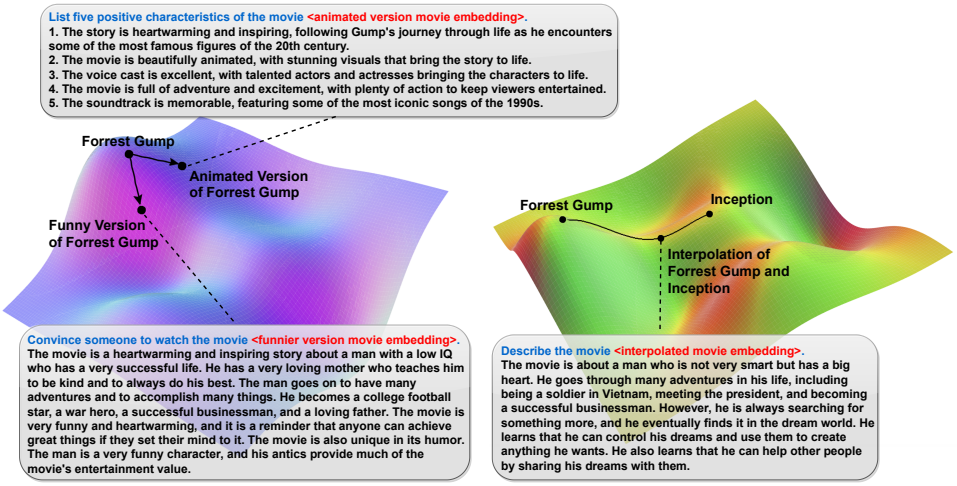

Figure 1: Decoded outputs of our model, ELM, for hypothetical movie embeddings. Text in blue and red correspond to the textual and domain embedding portions of the prompt to ELM. Text in black corresponds to the output of ELM.

be futile. However, a large language model (LLM) trained to *interpret the embedding representation itself* could handle such a task (see Fig. 1 for an illustration).

This paper introduces a novel framework to interpret *domain embeddings*[1] by leveraging the power of LLMs (Devlin et al., 2018; Liu et al., 2021; Google et al., 2023). Our method seamlessly introduces embeddings into LLMs by training *adapter layers* to map domain embedding vectors into the token-level embedding space of an LLM, which in turn allows one to treat these vectors as token-level encodings of the entities or concepts they represent. We train an LLM on a collection of tasks designed to facilitate the robust, generalizable interpretation of vectors in the domain embedding space. Our approach allows us to engage in a direct "dialogue" about these vectors, to query the LLM with intricate embedding data, and tease out narratives and insights from these dense vectors.

Our contributions are as follows. First, we formulate the problem of interpreting embeddings using LLMs. We then propose the *Embedding Language Model (ELM)*, a novel language model framework which, using trained adapters, can accept domain embedding vectors as parts of its textual input sequence to allow interpretation of continuous domain embeddings using natural language. We also develop a training methodology to fine-tune pretrained LLMs for domain-embedding interpretation. Finally, we test ELM by constructing 25 training tasks to allow interpretation of movie and user embeddings derived from the MovieLens 25M dataset (Harper & Konstan, 2015). We demonstrate our trained ELM on a variety of problems, including: generalizing CAVs as an interpretability method, describing hypothetical embedded entities, and interpreting user embeddings in a recommender systems by generating preference profiles. Our work bridges the gap between the rich data representations of domain embeddings and the expressive capabilities of LLMs.

2 Integrating Domain Embeddings with Large Language Models

We assume a *domain embedding* $E_D : \mathbb{V} \to \mathcal{W}$ which maps entities in some vocabulary $\mathbb{V}$ (e.g., a set of known users and items) into a latent space $\mathcal{W} \subseteq \mathbb{R}^n$, equipped with a distance metric $d : \mathcal{W} \times \mathcal{W} \mapsto \mathbb{R}_+$. The embedding vector $E_D(v)$ of any $v \in \mathbb{V}$ is typically a representation used in some downstream task, and is trained to encompass the latent features of v necessary for this task. For example, in a recommender system, *collaborative filtering* (CF) (Salakhutdinov & Mnih, 2007; Koren et al., 2011) is often used to generate embedding vectors of users and items s.t. the dot product (or cosine similarity) of $E_D(u)$ and $E_D(i)$ is predictive of user u's affinity for item i. In

[1]We use the term "domain embedding" to differentiate the external embeddings to be interpreted from those captured by an LLM at various layers of the network.

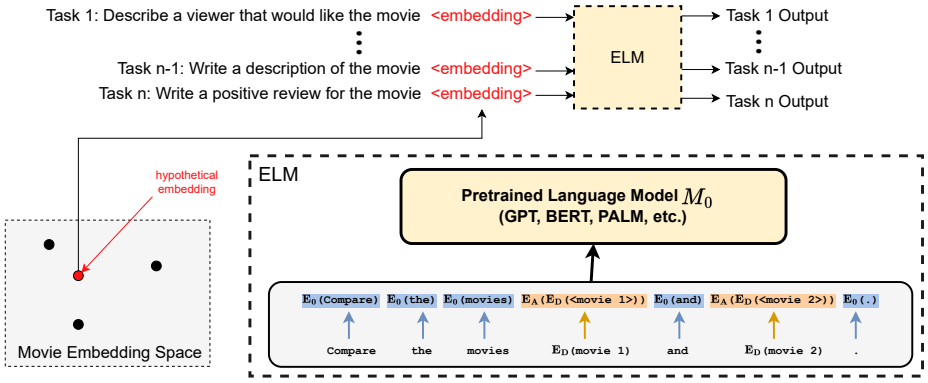

Figure 2: We train ELM using n tasks that incorporate embeddings as tokens in language. Specifically, to incorporate the domain embedding space $\mathcal{W}$, we enhance the pretrained LLM $\mathcal{M}$ with an adapter model $E_A : \mathcal{W} \mapsto \mathcal{Z}$ to create a new language model $\mathcal{M}_{\text{ELM}}$, ensuring tokens and embeddings are mapped to a shared space. While E_0 projects language tokens to $\mathcal{Z}$, the adapter E_A learns to project domain embedding vectors E_D from embedding space $\mathcal{W}$ to the same space, $\mathcal{Z}$. The resulting sequence is input to a pretrained language model M_0, which can be further fine-tuned (see Sec. 2).

Table 1: Glossary

$\mathcal{W}$	Domain embedding space	d	Domain space metric
$\mathcal{Z}$	Token (language) embedding space	$\mathcal{X}$	Token space
E_0	LLM embedding	$\mathcal{M}$	Pretrained LLM
E_A	Adapter ($\mathcal{W} \mapsto \mathcal{Z}$)	$\mathcal{M}_{\text{ELM}}$	Embedding Language Model
E_D	Domain embedding	M_0	LLM excluding input embedding layer

image classification, E_D might generate embedding representations useful for object detection (Zou et al., 2023), while in search/information retrieval, E_D might embed both queries and documents in $\mathcal{W}$ to measure a document's relevance to a query (Zuccon et al., 2015; Luan et al., 2021).

We assume a pretrained LLM, $\mathcal{M} : \mathcal{X}^H \mapsto \mathcal{X}^H$, which maps (length H) sequences of language tokens $x \in \mathcal{X}$ into another. Modern LLMs are composed of two parts, $\mathcal{M} = (E_0^H, M_0)$. The first part is *an embedding layer*, $E_0 : \mathcal{X} \mapsto \mathcal{Z}$ (not to be confused with the domain embeddings E_D), which maps tokens $x \in \mathcal{X}$ to their respective *token embedding* representations $z \in \mathcal{Z}$ (as distinct from the domain embedding). The second part is the *the dense model*, $M_0 : \mathcal{Z}^H \mapsto \mathcal{X}^H$, which maps a sequence of token embeddings to a sequence of tokens [2] (e.g., transformer, Vaswani et al. (2017)).

We now define the *Embedding Language Model (ELM)*, a framework which allows one to train a model using textual prompts mixed with continuous domain embedding vectors (see Fig. 2 for an illustration). ELM incorporates domain embedding vectors using *adapter layers* to interface with a text-only LLM. We note that we are not learning E_D itself, but rather wish to interpret it. In other words, given an embedding vector $E_D(v)$, we wish to train an LLM to meaningfully engage in discourse about the entity represented by $E_D(v)$ – even for a hypothetical entity v.

Problem Formulation. We assume access to a domain embedding space $(\mathcal{W}, d)$, and to pairs $(v, E_D(v))$ for training as explained below. Note that we do not assume access to the mapping E_D (except for empirical evaluation). Furthermore, our technique is agnostic to the type and source of $(\mathcal{W}, d)$. Finally, we assume access to a pretrained text-only LLM $\mathcal{M}$.

To interpret embedding vectors, we require some semantic information about the entities to which they correspond. To this end, let $\mathcal{T}$ be a set of *tasks*, where each task $t \in \mathcal{T}$ is specified by some mapping from joint input sequences $s \in (\mathcal{X} \cup \mathcal{W})^H$ to distributions over output sequences $o \in \mathcal{X}^H$.[3] Intuitively, a task t captures some form of semantic information about the entity represented by $w \in \mathcal{W}$, e.g., the plot of a movie, the preferences of a user, etc. The set of tasks $\mathcal{T}$ should be diverse enough to allow the ELM to extract semantic information from $\mathcal{W}$ to support interpretation, and

[2] In practice, the dense model outputs logits over tokens, which are sampled by a decoding procedure.

[3] More generally, we can define output sequences in $(\mathcal{X} \cup \mathcal{W})^H$ as well; we leave this for future work.